33 ASYNC / AWAIT / PROMISE — WHAT THEY ARE, AND WHERE THIS APP ACTUALLY USES THEM

theory/32 mentioned that every student.service.ts method is async and returns a Promise, "because MongoDB is a network call." This file slows that down and explains WHAT a Promise actually is, WHY JavaScript needs one at all, and then walks through every place in THIS app that uses (or deliberately doesn't use) the pattern.

THE PROBLEM THIS SOLVES

JavaScript runs on ONE thread (theory/20-single-thread-vs-multithreading.txt). If `this.studentModel.findById(id)` blocked that one thread until MongoDB replied over the network, your entire server would freeze — no other request, from any other user, could be handled — for however long that database round-trip takes.

So instead of blocking, a call like that returns IMMEDIATELY with a placeholder object, and the real work happens in the background while Node moves on to handle other requests.

A Promise IS THAT PLACEHOLDER

Think of it as a receipt/token, not the food itself:

```
this.studentModel.findById(id)
// returns, right now, this instant:
//   Promise { <pending> }
// NOT the student document — the document doesn't exist yet
// as far as this line of code is concerned.
```

A Promise is always in one of three states:

pending	→ still waiting (MongoDB hasn't replied yet)
fulfilled	→ got a result (the document, or null if not found)
rejected	→ something went wrong (network error, invalid query)

Nothing about a Promise blocks anything. It just sits there as an object you can attach "do this when you're done" instructions to.

await — HOW YOU ACTUALLY GET THE REAL VALUE OUT

```
const student = await this.studentModel.findById(id).exec();
```

`await` pauses ONLY the function it's written inside — not the whole app, not other requests being handled by other users at the same time. It says: "don't run the next line until this specific Promise settles, then unwrap it into the real value."

- If the Promise fulfills → `student` becomes the real document (or `null`, if nothing matched).

- If the Promise rejects → `await` throws, same as a normal `throw` — catchable with try/catch, or left to bubble up to Nest's exception filter (theory/24-exception-handling.txt).

Without `await`:

```
const student = this.studentModel.findById(id).exec(); // BUG
if (!student) { ... }   // this is checking a Promise OBJECT,
                        // which is always truthy — this branch
                        // NEVER runs, even when nothing was found
```

This is the single most common async bug: forgetting `await` doesn't crash anything, it just silently gives you the wrong thing to check.

async — THE LABEL THAT MAKES A FUNCTION await-CAPABLE

```
async findOne(id: string): Promise<StudentDocument> { ... }
```

`await` is only legal inside a function marked `async`. Marking a function `async` also means IT ITSELF now always returns a Promise, even if you `return student;` (a plain value) inside it — Nest/JS automatically wraps that return value in a fulfilled Promise. This is why the return type is written as `Promise<StudentDocument>`, not `StudentDocument` — that's not a lie about what studentService.findOne() gives back, it's the literal type of what `async` produces.

This makes async "contagious" upward: whatever calls an async function must itself either `await` it or handle it as a Promise — which is exactly why student.controller.ts's methods don't need to be `async` themselves in this app: they just `return this.studentService.findOne(id)` and hand the Promise straight to Nest, which awaits it for you before sending the HTTP response.

WHERE THIS APP ACTUALLY USES IT — student.service.ts

Every method that touches `this.studentModel` is async, because every one of them is a real MongoDB round-trip:

```
async create(dto: CreateStudentDto): Promise<StudentDocument> {
  const newStudent = new this.studentModel(dto); // in-memory, instant, no await needed
  return newStudent.save();                      // network call -> Promise
}

async findAll(): Promise<StudentDocument[]> {
  return this.studentModel.find().exec();        // network call -> Promise
}

async findOne(id: string): Promise<StudentDocument> {
  const student = await this.studentModel.findById(id).exec(); // must await to get the real doc/null
  if (!student) {
    throw new NotFoundException(`Student ${id} not found`);
  }
  return student;
}
```

Note `create()` doesn't `await newStudent.save()` — it just `return`s the Promise directly. That's fine and common: since the function is `async`, returning a Promise from it is automatically "flattened" into the same

Promise, rather than becoming a Promise-of-a-Promise. `await` is only strictly needed when you must DO something with the resolved value before returning (like the `if (!student)` check in findOne).

WHERE THIS APP DELIBERATELY DOESN'T USE IT — task.service.ts

```
findAll(): Task[] {
  return this.tasks;
}
```

No `async`, no Promise, no `await`. `this.tasks` is a plain array already sitting in memory (theory/32) — there is no waiting to do, so adding `async` here would be pure overhead: it would force every caller to `await` a value that was already available instantly. This is the actual decision rule for your own code going forward:

```
Reading/writing something already in memory (a local array, a
simple calculation)          -> stay synchronous, no async/await.
```

> Reading/writing anything that leaves the process (MongoDB, an external HTTP API, the filesystem) → async + await, always.

HOW IT FLOWS END-TO-END FOR ONE REQUEST — POST /student/createstudent

1. Controller: create(@Body() dto) { return this.studentService.create(dto); } - Not async itself — it just gets back a Promise and hands it straight to Nest.
2. Nest internally awaits that Promise before writing the HTTP response — this is built into how Nest handles any controller method's return value, whether it's a plain value or a Promise.
3. Inside create(): `new this.studentModel(dto)` builds the document in memory (instant, validated against @Prop() rules from theory/31) — no network yet.
4. `.save()` is called and its Promise is returned. THIS is the actual network round-trip to your MongoDB Atlas cluster (theory/30-mongodb-connection.txt) — bytes go out, MongoDB inserts the document, bytes come back with the saved doc (_id, createdAt, updatedAt).
5. Once that Promise fulfills, Nest sends the resolved document back to the client as the JSON response.

At every one of these steps, the Node process is FREE to handle other incoming requests from other users — nothing here blocks the thread; only this one request's own logical flow is "paused" waiting on its own Promise.

★ RULE OF THUMB

A Promise is a placeholder for a value that isn't ready yet, because getting it involves leaving the current process (network/disk/DB). `await` pauses just the current function until that specific Promise settles, then swaps it out for the real value (or throws, if it rejected). `async` is the label a function needs to be allowed to use `await` inside it, and it makes that function itself always return a Promise. Use async/await for anything touching the database or an external service; skip it entirely for plain in-memory logic — adding it there buys you nothing and just adds unnecessary Promise-unwrapping for every caller.

See also: **theory/32-injecting-mongoose-models.txt**, **theory/30-mongodb-connection.txt**, **theory/20-single-thread-vs-multithreading.txt**, **theory/24-exception-handling.txt**